

FACIAL EMOTION RECOGNITION

Final presentation
Group 1

PREVIOUS PRESENTATION RECAP

- In the last presentation, we decide to continue exploring 2 different approaches: Custom model construction and transfer learning.
- As our models get more and more complex → Colab's limit reached multiple time.

==> We decided to stop the manual route entirely and focused on transfer learning.

PROBLEM RECAP

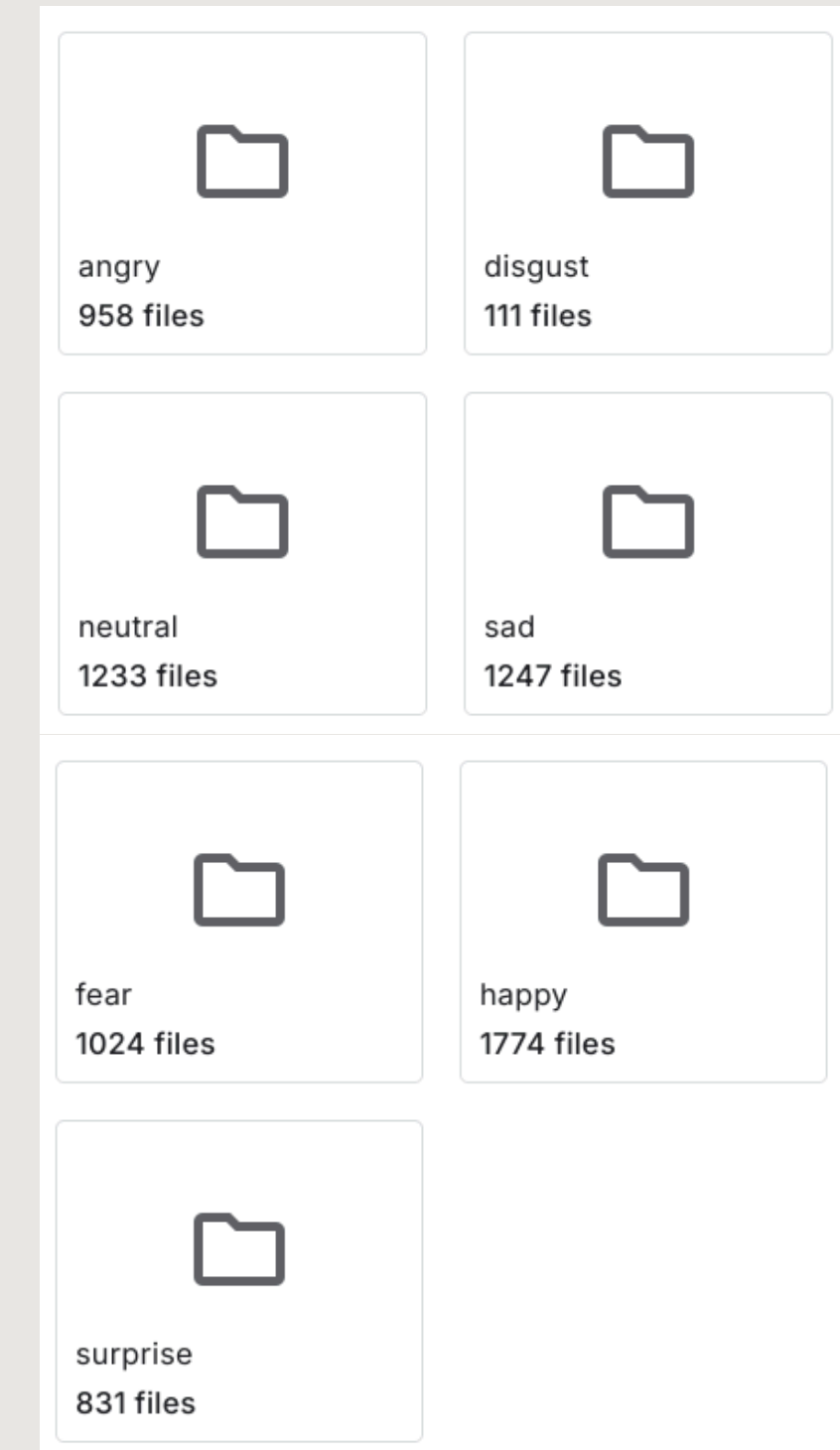
Goal: Obtain a 70-80% accuracy on the FER-2013 facial emotion dataset.

Challenges

- Our task requires the use of the FER-2013, which is an imbalance and inaccurate dataset.
- Google Colab 2 hours session limit → Cannot use bigger, more complex models.

BASELINE FOR IMPROVEMENTS

- FER-2013 is a noisy and unbalance dataset, with the two most-unbalanced labels are ‘happy’ and ‘disgust’.
- Therefore, we have decided to sacrifice the ‘disgust’ classification to improve performance.
- Data augmentation is also used to clone existing images to workable data.



BASELINE FOR IMPROVEMENTS

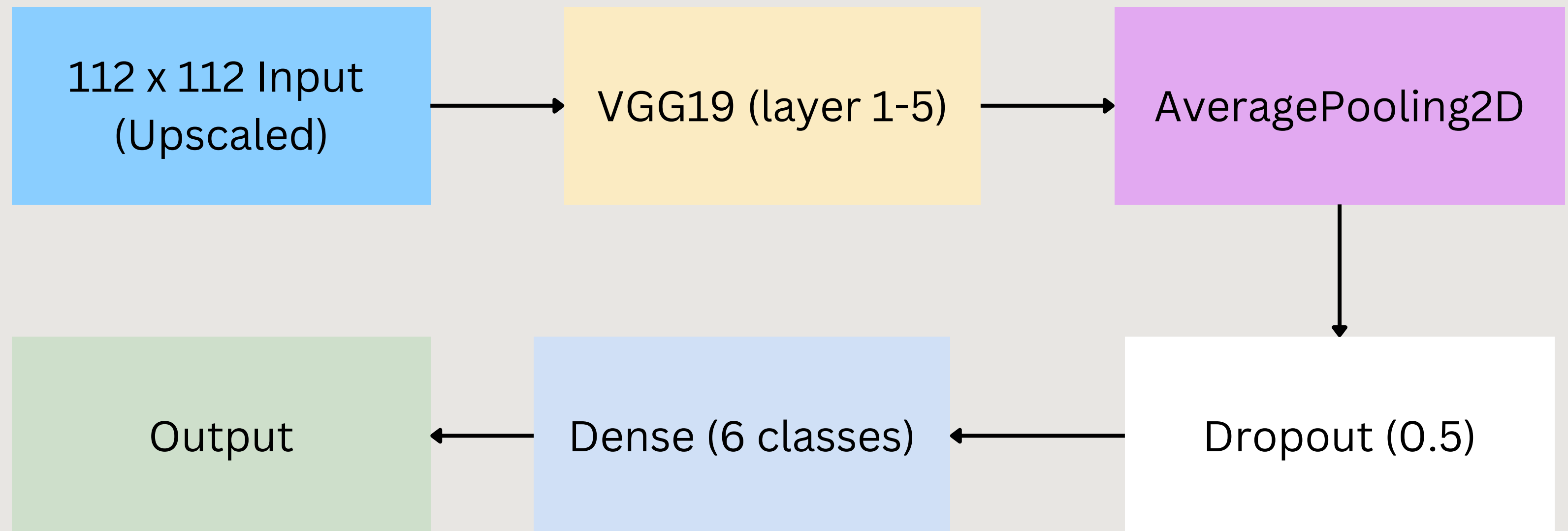
- Our last transfer learning uses ConvNeXt-Tiny, which while being faster, it natively expects 224x224, RGB images. With our dataset being 48x48, it can lead to feature collapse due to low resolution.
- Solution: switch to another model that is more low-resolution-friendly.
- Our choice: **VGG-19**.

BASELINE FOR IMPROVEMENTS

- During this time, we have also experimented with layer freezing/unfreezing.
- For this feature to work, the parameters must be precisely fine-tuned:
 - Too little freezing → the model cannot learn much.
 - Too much → the original model gets massively altered, remove the benefit of transfer learning (time and resources).

First iteration of the VGG19 Transfer Learning Model

First VGG19 Transfer Learning Model - Model Structure



First VGG19 Transfer Learning Model - Model Structure

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 112, 112, 3)	0
block1_conv1 (Conv2D)	(None, 112, 112, 64)	1,792
block1_conv2 (Conv2D)	(None, 112, 112, 64)	36,928
block1_pool (MaxPooling2D)	(None, 56, 56, 64)	0

block2_conv1 (Conv2D)	(None, 56, 56, 128)	73,856
block2_conv2 (Conv2D)	(None, 56, 56, 128)	147,584
block2_pool (MaxPooling2D)	(None, 28, 28, 128)	0

block3_conv1 (Conv2D)	(None, 28, 28, 256)	295,168
block3_conv2 (Conv2D)	(None, 28, 28, 256)	590,080
block3_conv3 (Conv2D)	(None, 28, 28, 256)	590,080
block3_conv4 (Conv2D)	(None, 28, 28, 256)	590,080
block3_pool (MaxPooling2D)	(None, 14, 14, 256)	0

block4_conv1 (Conv2D)	(None, 14, 14, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block4_conv4 (Conv2D)	(None, 14, 14, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 7, 7, 512)	0

block5_conv1 (Conv2D)	(None, 7, 7, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 7, 7, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 7, 7, 512)	2,359,808
block5_conv4 (Conv2D)	(None, 7, 7, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 3, 3, 512)	0

global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0
dropout (Dropout)	(None, 512)	0
dense (Dense)	(None, 6)	3,078

First VGG19 Transfer Learning Model - Strategy

Preprocessing: Data augmentation is used to pad the classes so that every labels have the same number of images to 'happy'.

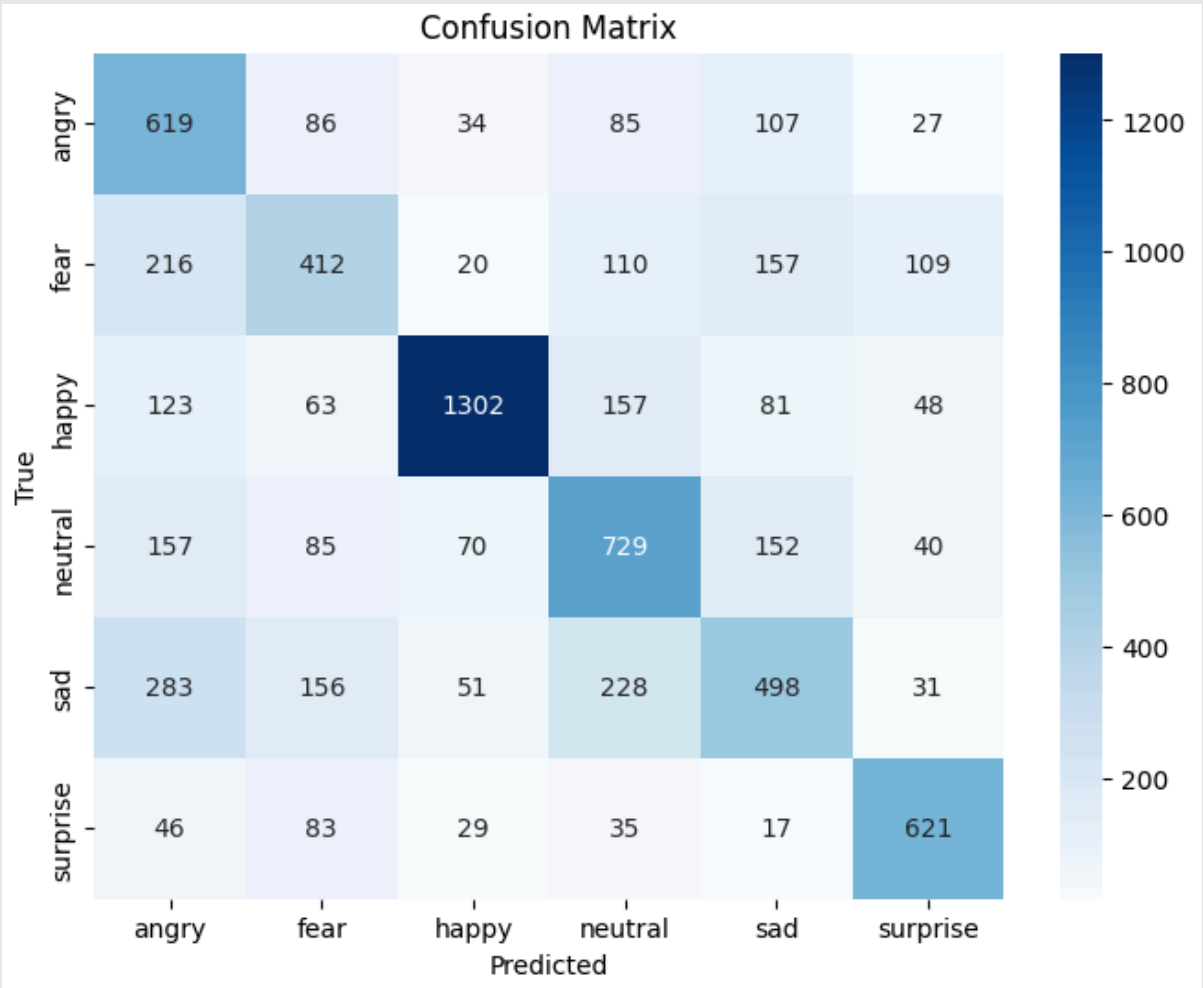
The training strategy for this model shall be executed as follows:

- The first 10 epochs: the entire VGG19 model would be frozen, allows us to train the classification heads section of the model.
- The next 50 epochs: unfreeze layer 5 of the VGG19, lower the learning rate from the first run. This layer includes early stopping and dynamic learning rate adjustment to combat overfitting.

First VGG19 Transfer Learning Model - Result

The performance of this model is presented with the below charts:

	precision	recall	f1-score	support
angry	0.43	0.65	0.52	958
fear	0.47	0.40	0.43	1024
happy	0.86	0.73	0.79	1774
neutral	0.54	0.59	0.57	1233
sad	0.49	0.40	0.44	1247
surprise	0.71	0.75	0.73	831
accuracy			0.59	7067
macro avg	0.58	0.59	0.58	7067
weighted avg	0.61	0.59	0.59	7067

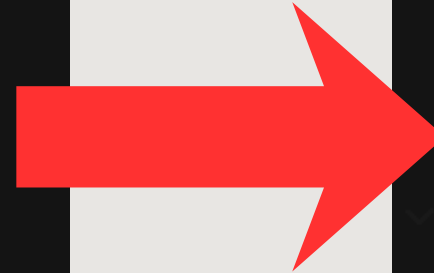


We see and immediate uplift in performance. For our training run, the model achieved 69% on the train data split and 59% on the evaluation data split on the 18th epoch.

Final VGG19 Transfer Learning Model

Final VGG19 Transfer Learning Model

```
✓ train_datagen = keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255, # Add this for normalization if not already present  
    validation_split=0.2,  
)  
  
✓ train_generator = train_datagen.flow_from_directory(  
    'train',  
    target_size=(112, 112), # Changed target_size to 224x224  
    color_mode='rgb',       # Changed color_mode to rgb  
    class_mode='categorical',  
    batch_size=64,  
    subset='training'  
)
```

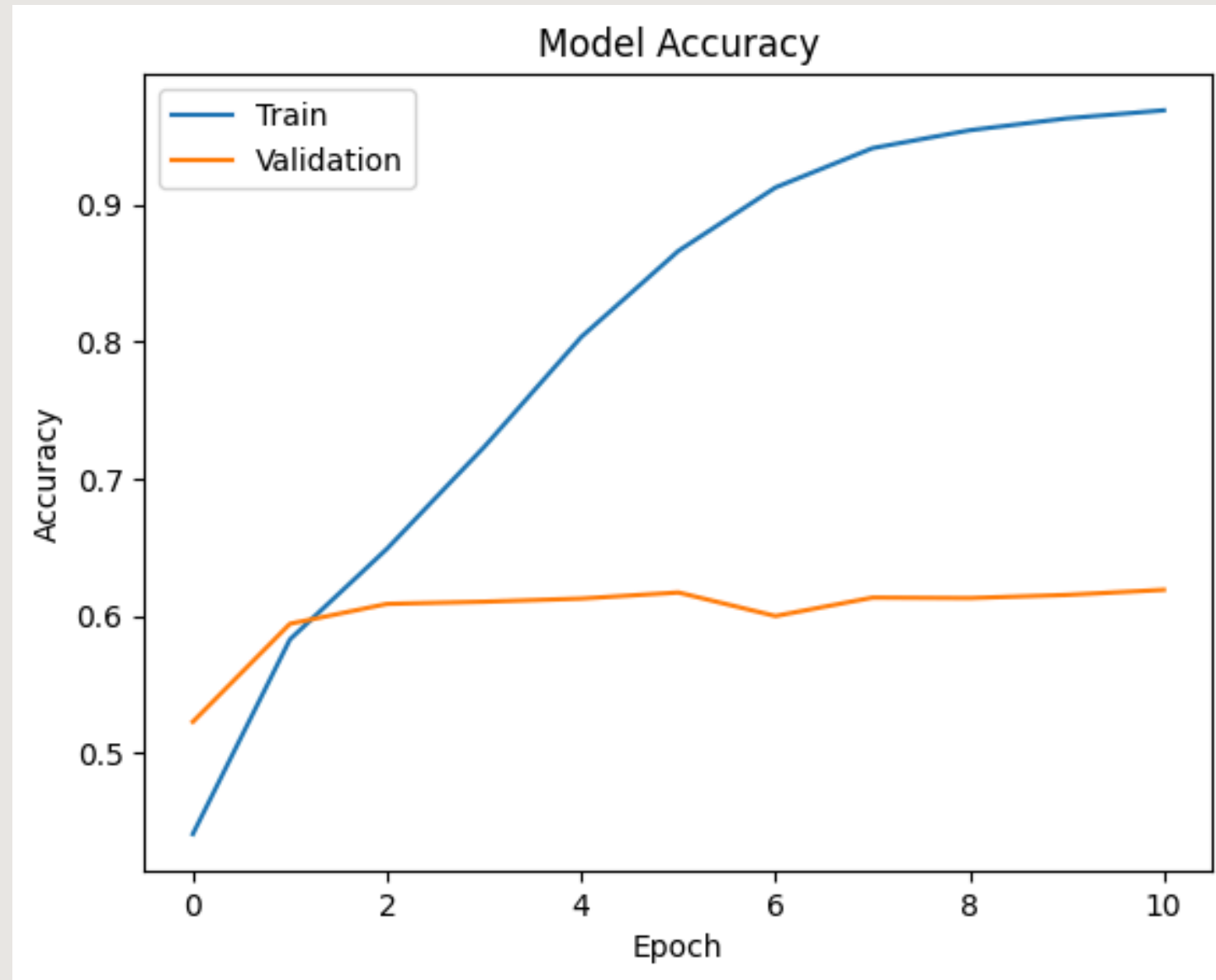


```
train_datagen = keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255, # Add this for normalization if not already present  
    validation_split=0.2,  
    rotation_range = 15,  
    width_shift_range = 0.15,  
    height_shift_range = 0.15,  
    shear_range = 0.15,  
    zoom_range = 0.15,  
    horizontal_flip = True  
)  
  
train_generator = train_datagen.flow_from_directory(  
    'train',  
    target_size=(112, 112), # Changed target_size to 112x112  
    color_mode='rgb',       # Changed color_mode to rgb  
    class_mode='categorical',  
    batch_size=32, Change batch size to 32  
    subset='training'  
)
```

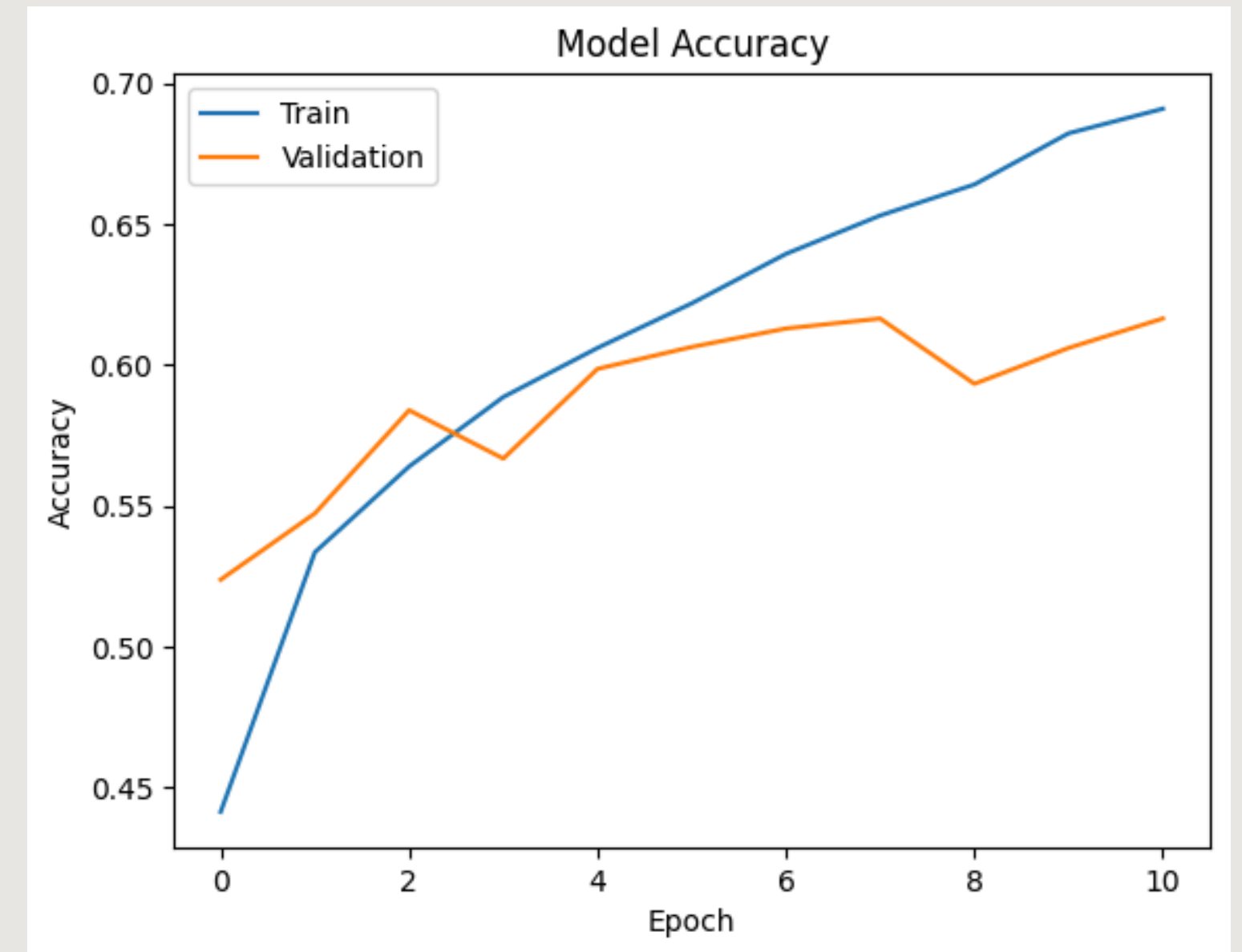
New

- Old version use offline augmentation
- New version use online augmentation helps reduce overfit
- Batch size of 32 increase accuracy by 0.5-1 % compare to batch size of 64

Final VGG19 Transfer Learning Model



Use offline augmentation (OLD version)



Use online augmentation (FINAL version)

Final VGG19 Transfer Learning Model

OLD

Training phase 1:

- + epochs = 10
- + learning rate = $1e-3$
- + All VGG19 layers is frozen

Training phase 2:

- + epochs = 40
- + learning rate = $1e-5$
- + Block 5 is unfrozen

NEW

Training phase 1:

- + epochs = 10
- + learning rate = $1e-3$
- + All VGG19 layers is frozen

Training phase 2:

- + epochs = 10
- + learning rate = $1e-4$
- + Block 5 is unfrozen

Training phase 3:

- + epochs = 35
- + learning rate = $5e-6$
- + All VGG19 layers is unfrozen

Final VGG19 Transfer Learning Model

```
- val_accuracy: 0.6475 - val_loss: 1.0166  
- val_accuracy: 0.6454 - val_loss: 1.0293  
- val_accuracy: 0.6482 - val_loss: 1.0463  
- val_accuracy: 0.6505 - val_loss: 1.0649
```

```
lr_scheduler = ReduceLROnPlateau(  
    monitor='val_loss',  
    factor=0.2,    # Drop learning rate to 2  
    patience=2,    # Trigger this after just  
    min_lr=1e-7  
)  
  
#The Auto-Cutoff (The Kill Switch)  
early_stopping = EarlyStopping(  
    monitor='val_loss', # Always monitor lo  
    patience=5,        # Wait 5 epochs bef  
    verbose=1,  
    restore_best_weights=True # CRITICAL: R  
)
```

OLD

NEW

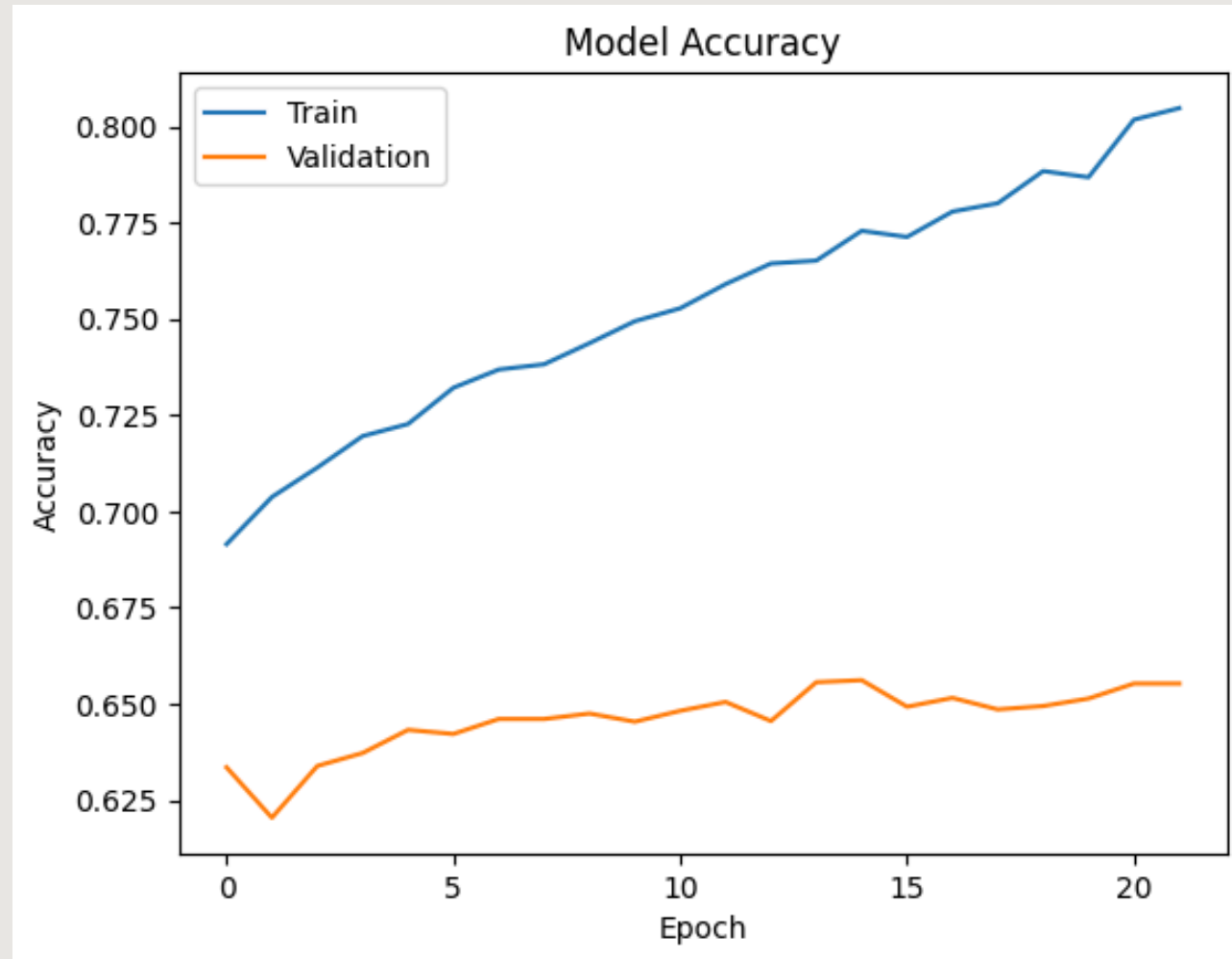
```
lr_scheduler = ReduceLROnPlateau(  
    monitor='val_accuracy',  
    factor=0.2,    # Drop learning rate to 20% of  
    patience=5,    # Trigger this after just 2 s  
    min_lr=1e-7  
)  
  
#The Auto-Cutoff (The Kill Switch)  
early_stopping = EarlyStopping(  
    min_delta = 0.00005,  
    monitor='val_accuracy', # Always monitor lo  
    patience=5,            # Wait 5 epochs before  
    verbose=1,  
    restore_best_weights=True # CRITICAL: Revert
```

NEW

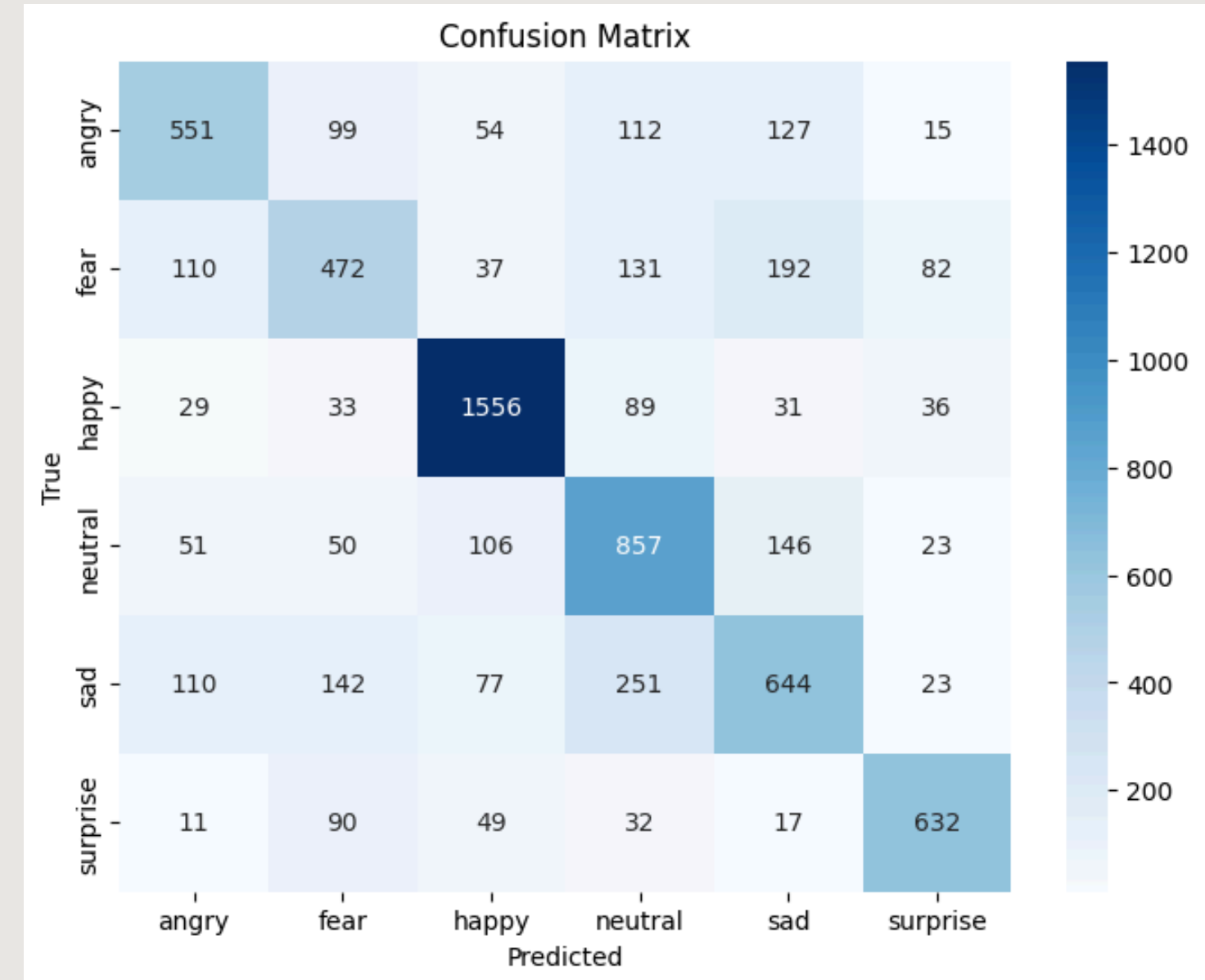
```
early_stopping = tf.keras.callbacks.EarlyStopping(  
    monitor='val_accuracy',  
    patience=7,  
    restore_best_weights=True # Reverts if Block 4 ca  
)  
  
lr_scheduler = ReduceLROnPlateau(  
    monitor='val_accuracy',  
    factor=0.4,    # Drop learning rate to 20% of its  
    patience=5,    # Trigger this after just 2 stalled  
    min_lr=1e-7  
)
```


Final VGG19 Transfer Learning Model

Result on test set:



Phase 3 Training and Validation Accuracy



	precision	recall	f1-score	support
angry	0.64	0.58	0.61	958
fear	0.53	0.46	0.49	1024
happy	0.83	0.88	0.85	1774
neutral	0.58	0.70	0.63	1233
sad	0.56	0.52	0.54	1247
surprise	0.78	0.76	0.77	831
accuracy			0.67	7067
macro avg	0.65	0.65	0.65	7067
weighted avg	0.66	0.67	0.66	7067

Cross-version Performance Comparasion

```
Epoch 14/15
359/359 ————— 134s 375ms/step - accuracy: 0.5182 - loss: 1.2848 - val_accuracy: 0.5123 - val_loss: 1.2805
Epoch 15/15
359/359 ————— 132s 369ms/step - accuracy: 0.5215 - loss: 1.2762 - val_accuracy: 0.5192 - val_loss: 1.2752
```

Last presentation's model



Change of model + Augmented data
+ 'disgust removal'

```
Epoch 17/50
544/544 ————— 108s 199ms/step - accuracy: 0.6723 - loss: 0.8496 - val_accuracy: 0.5824 - val_loss: 1.0898
Epoch 18/50
544/544 ————— 108s 198ms/step - accuracy: 0.6907 - loss: 0.7956 - val_accuracy: 0.5948 - val_loss: 1.0740
Epoch 19/50
```

1st VGG19 transfer learning model



Improved data augmentation + new
training strategy

```
Epoch 40/55
707/707 ————— 158s 223ms/step - accuracy: 0.8018 - loss: 0.5364 - val_accuracy: 0.6553 - val_loss: 1.1321
Epoch 41/55
707/707 ————— 158s 223ms/step - accuracy: 0.8048 - loss: 0.5261 - val_accuracy: 0.6553 - val_loss: 1.1387
```

Final VGG19 transfer learning model

POSSIBLE FUTURE IMPROVEMENTS

- **This project:** Experimenting with K-fold cross-validation.
- **Expansion:** Use a different dataset altogether (AffectNet or RAF-DB)

REFERENCES

- Enes Ozturk, “Facial Emotion Recognition I VGG19 - FER2013”, Kaggle.
Available:<https://www.kaggle.com/code/enesztrk/facial-emotion-recognition-vgg19-fer2013/notebook>
- Manas Sambare, “FER-2013”, Kaggle.
Available:<https://www.kaggle.com/datasets/msambare/fer2013>
- Code snippets generated by Google Gemini

THANK YOU FOR LISTENING

Any questions for us?